ML@NOVA:

  &  王 &    ?

Multiple myeloma survival

Team identification

Name 1: Edgar Cucubica

Number 1: 67090

Name 2: Joana Wang

Number 2: 60225

Name 3: Renato Viola

Number 3: 60665

Final score: Public = 2.87235 | Private = 3.09368

Leaderboard ranking: 46º

Task [1.1]

From the bar plot on the right, we visualize the distribution of missing values across the features. These are our observations:

- There are no missing values in columns: *Age*, *Gender*, *Stage*, *TreatmentType* and *Censored*;
- Some features have missing values, namely: *GeneticRisk*, *ComorbidityIndex* and *TreatmentResponse*;
- There are 400 rows in total, 240 of which are labeled, 160 are unlabeled.

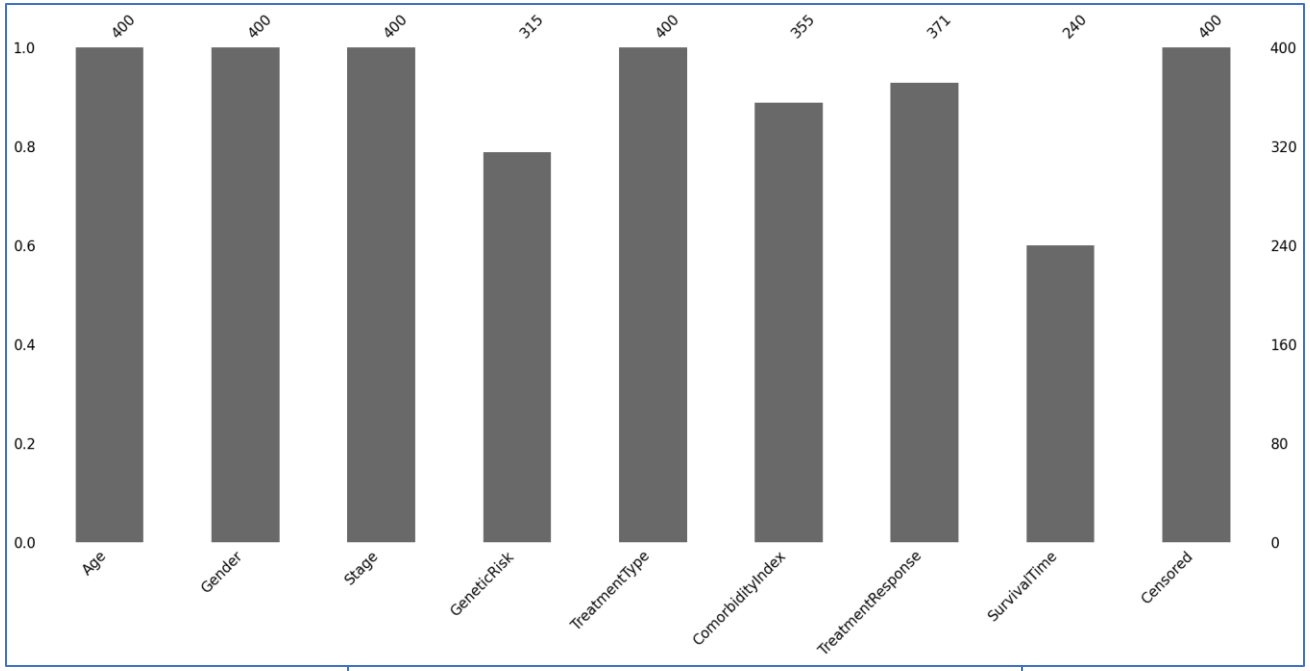


Fig 1 - Bar plot of feature missing values

Task [1.1]

From the heatmap on the right, we visualize the relationships in missingness between features. These are our observations:

- Generally, the features share a low missingness correlation between them, as the values presented are close to 0;
- In terms of missingness, the *SurvivalTime* variable is barely correlated with *GeneticRisk* and does not meaningfully correlate with *ComorbidityIndex* and *TreatmentResponse*.

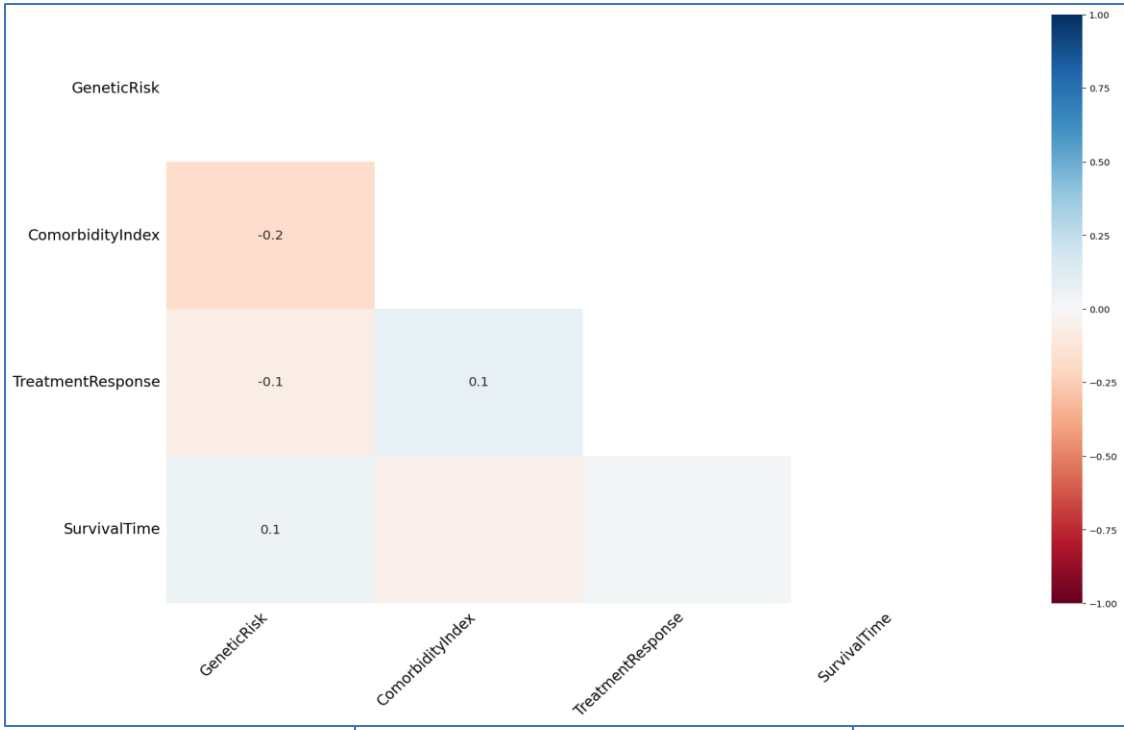


Fig.2 - Missingness Heatmap

Task [1.1]

From the matrix on the right, we may analyze the sparsity and patterns of missing data between variables, as non-missing values are represented by a dark color, while missing values are represent by a lighter one.

We will just be focusing on these patterns for now.

These are our observations:

- Overall, the missing data seems to be randomly distributed in each column;
- However, in certain columns like *GeneticRisk* and *SurvivalTime*, there seem to be points of greater concentration of missing data, represented by white blocks in the matrix;
- In a row, it is common for missing values to occur across several columns at once.

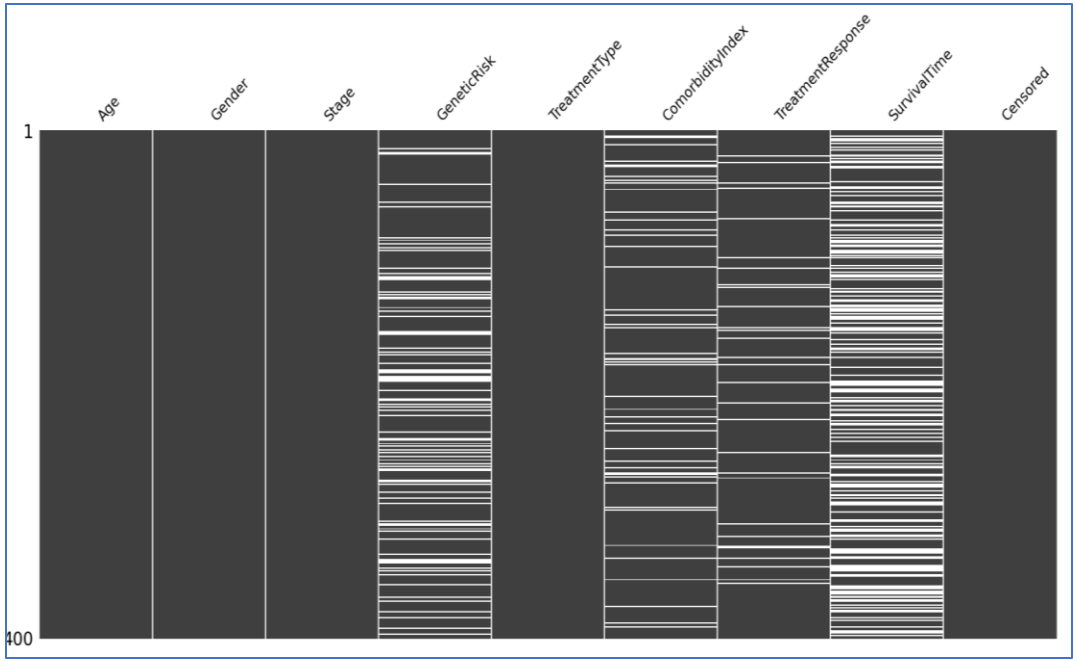


Fig.3 - Missing values matrix

Task [1.1]

From the dendrogram on the right, we identify the groups of columns with more similar missingness patterns. These are our observations:

- Columns with no missing data are all placed in the same horizontal line;
- The variables with missing data are vertically further apart from the initial horizontal line (in which there are no missing values) the more values are missing;
- In terms of missingness, *TreatmentResponse* is more closely related to *ComorbidityIndex* than either *GeneticRisk* or *SurvivalTime* are.

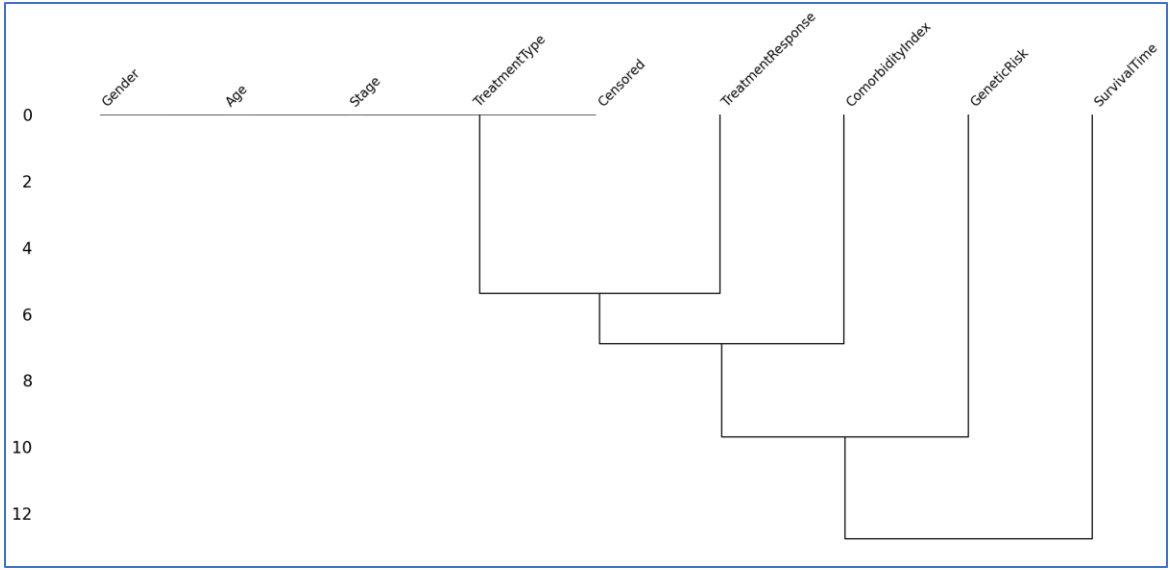


Fig.4 - Missing values dendrogram

Task [1.1]

Data preparation and validation pipeline:

- Upon analyzing the dataset, we dropped all the feature columns where there was missing data as well as any row that was censored. We were then left with 161 data points to work with. The pair plot analyzing these remaining data points can be found in the next slide.
- The feature matrix X, contains as columns: | **Age** | **Gender** | **Stage** | **TreatmentType** |.
- The target vector y contains the *SurvivalTime* and there is also a vector c for holding the *Censored* flags.
- After "cleaning" the dataset of missing values and censored rows, there are only 161 points remaining, as mentioned previously. As the dataset is already quite small, we wanted to keep at least 80% of it for training, equating to about 129 data points. Were we to use a train-validation-test split of 80/10/10, both the validation and test sets would have only 16 points each. Instead, we settled on a train-test split of 80/20 with 10-fold cross-validation on the training set, and we maintained this strategy up to the final tasks of the assignment. Our intent was to maximize data utilization for both training and validation, while also getting a decent estimate of a model's generalization performance, since cross-validation tests on each of the folds, averaging the performance across them.
- The metric used to evaluate the models developed along the assignment is the censored Mean Squared Error, which we will just refer to as cMSE from now on.

Task [1.1]

Pair plot analysis over features with missing data and censored rows:

*How does each column associate to the **SurvivalTime**?*

- **Age**: The density of points seems to be the highest at lower to mid-range survival times, as the scatter tends more to the left, especially for older ages, which seem to correlate to slightly shorter survival times;
- **Gender**: Both genders seem to display relatively similar distributions of survival times;
- **Stage**: Higher stages seem to tend more to the left, correlating with shorter survival times;
- **TreatmentType**: The distribution of both treatment types is not much different, but treatment type 0 tends more to the left, thus being associated with lower survival times.

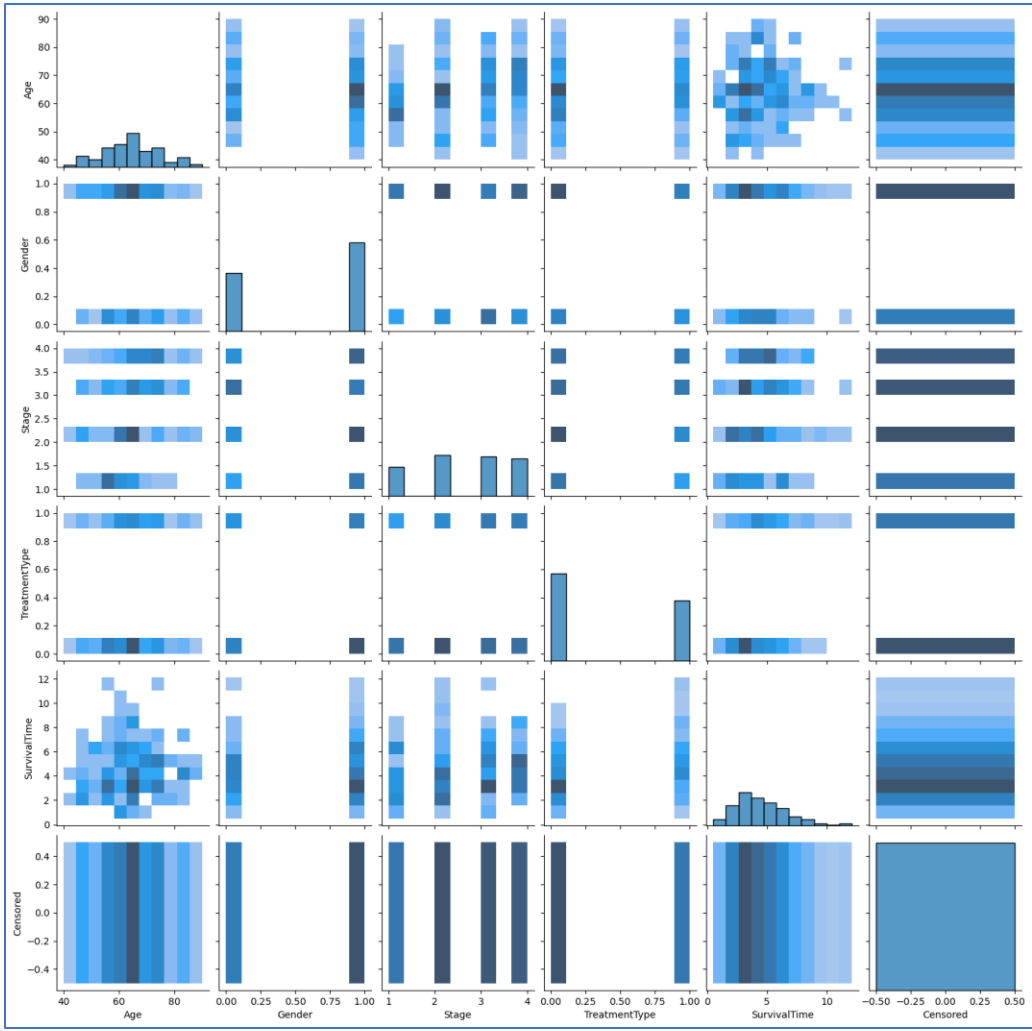
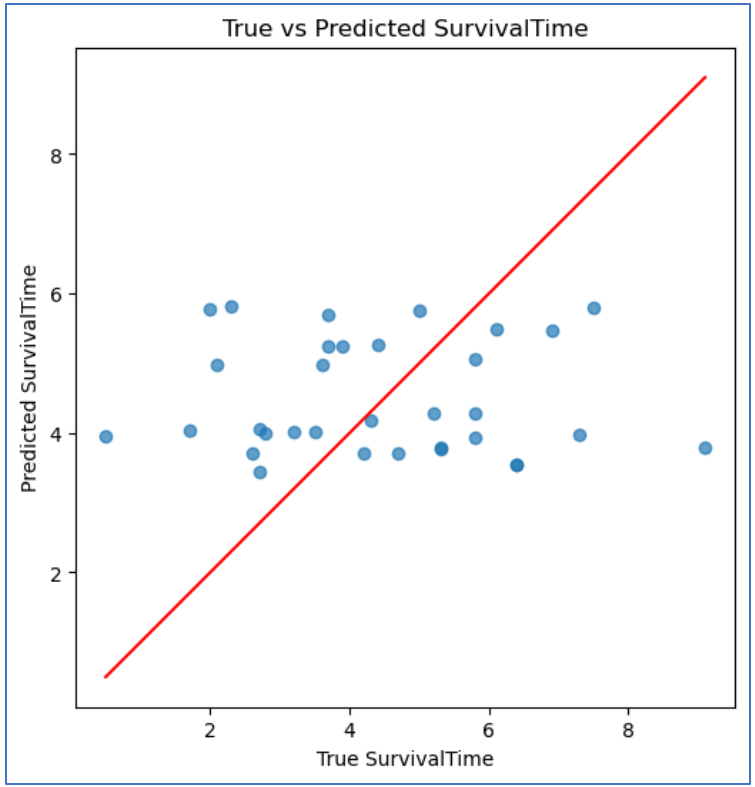


Fig.5 - Pair plot

Task [1.2]

Test cMSE	Kaggle cMSE
4.35506	3.44036



Test y-y hat plot – linear regression

Analyzing the y-y hat plot:

From the test y-y hat plot, it is clear to see that while the model does capture some information, the points deviate very noticeably from the line, and some predictions form small clusters while there are some outliers as well. Prediction errors are overall quite high.

As for the cMSE comparison, the Kaggle cMSE is significantly lower than that of the local test set. We theorize that this might be because our local test split most likely does not represent the data distribution properly, which we believe is mainly due to lack of data, rather than an imbalanced split.

Kaggle's test set may also just align better with our local training set, than our test set does.

Task [1.3]

$$cMSE = \sum_{n=1}^{N} (1 - c_n)(y_n - \hat{y}_n)^2 + c_n(\max(0, y_n - \hat{y}_n))^2.$$

Calculating the derivative of the cMSE loss:

$$cMSE = \sum_{n=1}^{N} (1-c_n)(y_n-\hat{y}_n)^2 + c_n(\max(0, y_n-\hat{y}_n))^2$$

- $C=0$,

$$\frac{\partial}{\partial \hat{y}_n}(y_n-\hat{y}_n)^2 = -2(y_n-\hat{y}_n)$$

- $C=1$,

$y_n - \hat{y}_n > 0$, $\frac{\partial}{\partial \hat{y}_n}(y_n-\hat{y}_n)^2 = -2(y_n-\hat{y}_n)$

$y_n - \hat{y}_n \leq 0$, $\frac{\partial}{\partial \hat{y}_n} 0 = 0$

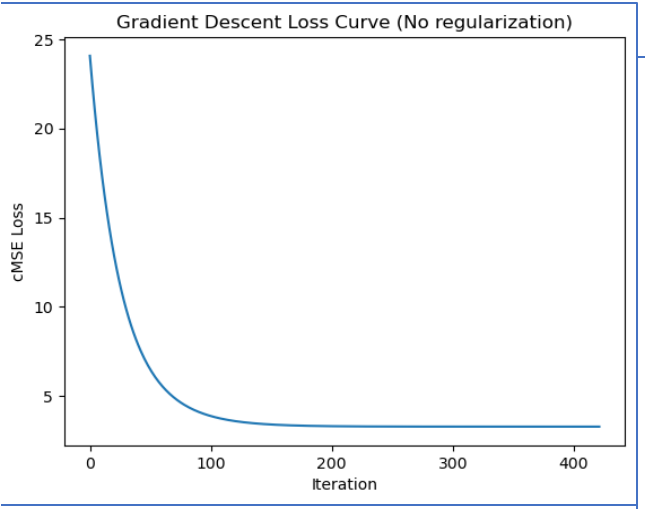
$$\frac{\partial cMSE}{\partial \hat{y}_n} = \sum_{n=1}^{N} [(1-c_n)\cdot(-2)\cdot(y_n-\hat{y}_n) + c_n \cdot (-2) \cdot \max(0, y_n-\hat{y}_n)] =$$

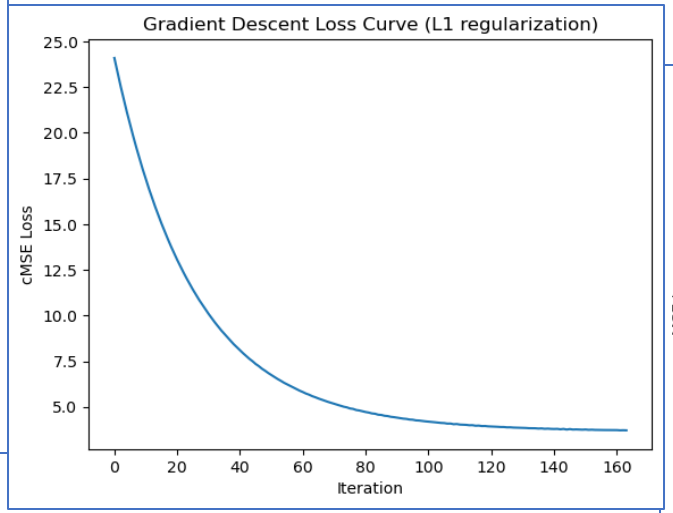
$$= -2 \sum_{n=1}^{N} [(1-c_n)(y_n-\hat{y}_n) + c_n \max(0, y_n-\hat{y}_n)]$$

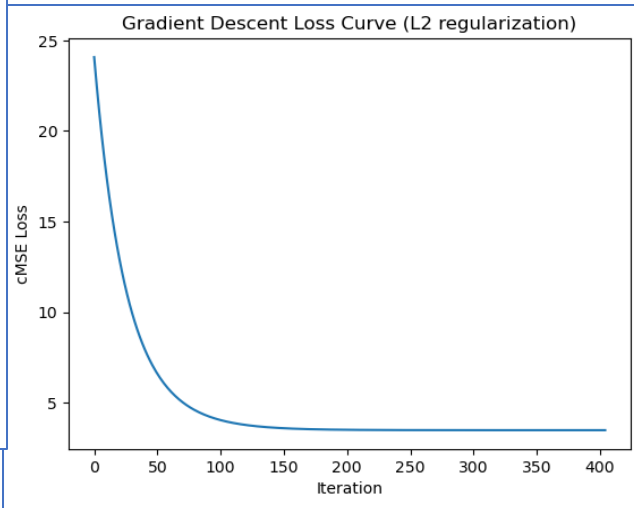
Task [1.3]

Regularization	Test cMSE	Kaggle cMSE
None	2.85420	3.20607
L1 (α=0.01)	3.24907	-
L2 (α=0.01)	2.93364	-

Loss curves for gradient descent

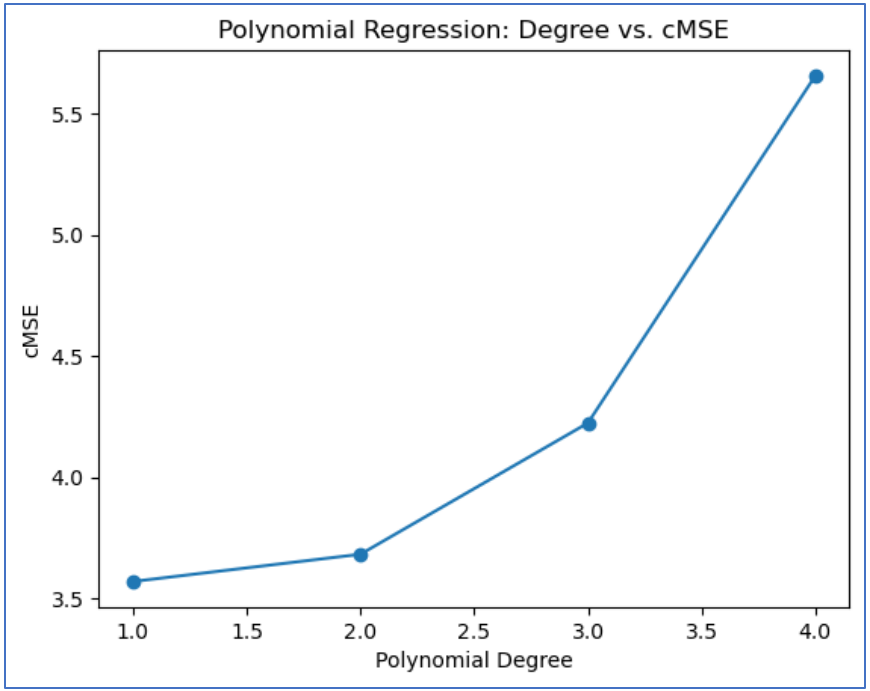


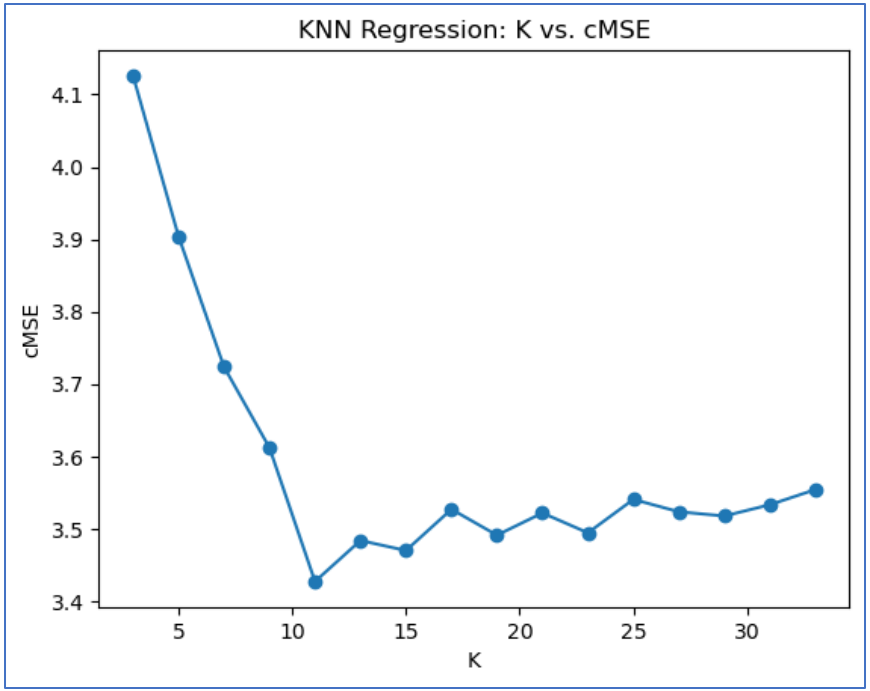




Task [2.1]

Using all labeled data, both censored and uncensored, we've determined the optimal values for the degree of polynomial features and for K in the KNN regression, using cross-validation. As can be observed in the following plots, the cMSE increases with the polynomial degree, even with the lower ones. The best value for K is 11.





Task [2.2]

Evaluating the models against the baseline:

As can be observed in the following table, displaying the various error metrics for each model, all models trained with task 2's data, significantly outperform the baseline in mean error, standard deviation and test score, indicating more predictive capability.

Out of these models, a simple linear regression is performing the most, at least in terms of mean error and test score. However, both the linear regression and KNN regression on task 2's data performed much worse than the baseline on Kaggle, despite showing a much more promising local test score.

It seems to us that the models trained on task 2's data may have either overfit the training data or cannot adapt well to the Kaggle test set. In the case of the linear regression, we believe the latter to be the case. We didn't even submit the polynomial model to Kaggle as we had already seen the error only go up with the degree, but we suspect it would overfit. As for the KNN regression, we strongly believe it is a case of having overfit the local training set.

As ironic as it is, our "best" model for this task is the linear regression.

Model	Max	Min	Mean	Std dev	Test score	Kaggle
Baseline	5.309497	0.122906	1.739313	2.086137	4.35506	3.44036
Linear	6.952703	0.004346	1.275022	1.822654	2.94767	3.74508
Poly (d=2)	5.917725	0.025293	1.421378	1.928217	3.32542	-
KNN (k=11)	5.609091	0.036364	1.469697	1.948771	3.29741	4.52088

Task [3.1]

We tried 3 different imputers:

- Simple imputer with 3 strategies: *mean*, *median* and *most frequent* - these last two returned subpar results compared to *mean*, so we dropped them and just went with the *mean*;
- KNN imputer for which we found that with cross-validation, the optimal value for K is 19;
- Iterative imputer.

Evaluation against the baseline:

- Without imputation, as expected, the baseline model has a significantly higher mean error and standard deviation than that of the models trained with the data from task 3.1;
- All the implemented imputation techniques are a great improvement over the baseline results, displaying more accuracy.
- The three imputation techniques, when evaluated with a linear regression model, are very close to each other in terms of results.

Imputation	Max error	Min error	Mean error	Std Dev
None	5.309497	0.122906	1.739313	2.086137
Simple Mean	6.075789	0.084432	1.280692	1.682250
KNN (k =19)	6.050910	0.035233	1.296114	1.693560
Iterative	6.075599	0.030936	1.283452	1.686608

Task [3.1]

Analyzing the y-y hat plot:

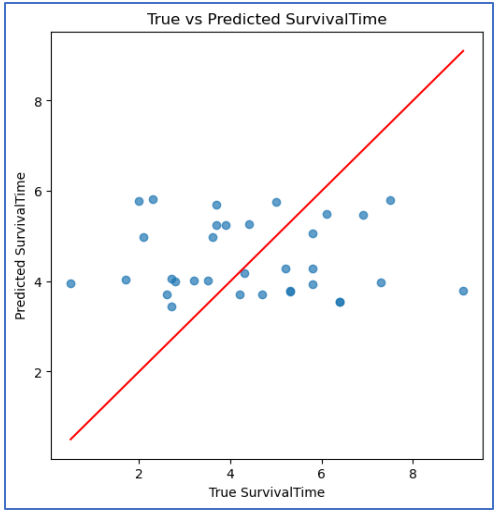
Since all the imputation techniques yielded similar results with a linear regression, we're only displaying here the plot of one of them, the Simple Mean imputation.

The baseline's plot shows the data points quite scattered and with a significant deviation from the red line, predictions are very innacurate.

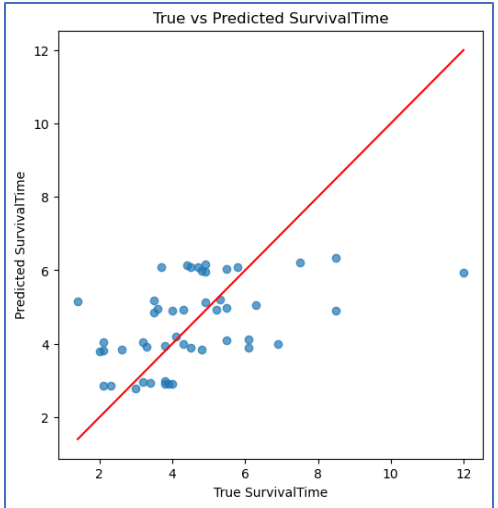
On the other hand, the simple mean imputation plots are more clustered and centered along the diagonal line.

As for the linear regression on the imputed dataset, quite a few predictions are still off.

However, these are still much more accurate and nuanced than the baseline's, indicating that the model is capable of capturing some patterns and variability in the target variable.



Baseline plot



Simple Imputated plot

Task [3.3]

Evaluation against the baseline:

Without imputation of the missing data, every tree-based and boosting model significantly outperformed the baseline model both in mean error and standard deviation;

Boosting models display the lowest mean errors and standard deviation, compared not only to the baseline but also to the conventional tree-based models. As such, these were the most performant models.

Model	Max error	Min error	Mean error	Std Dev	Test score	Kaggle
Baseline	5.309497	0.122906	1.739313	2.086137	4.35506	3.44036
Decision Tree	5.632353	0.086364	1.284895	1.752263	2.43448	3.35412
Random Forest	5.407375	0.014066	1.325817	1.756655	3.04435	3.45574
XGBoost	5.293990	0.060381	1.249277	1.650778	2.49613	3.30097
CatBoost	5.418007	0.055955	1.273691	1.727053	2.52150	3.14944
Histogram GB	5.461786	0.001279	1.281798	1.712220	2.52426	-

Task [3.3]

Evaluating the y-y hat plot against the baseline:

Imputing the dataset with a KNN imputation of K=19 was the one which yielded the best results in terms of cross-validation score, and as such, the one we adopted from this task onward.

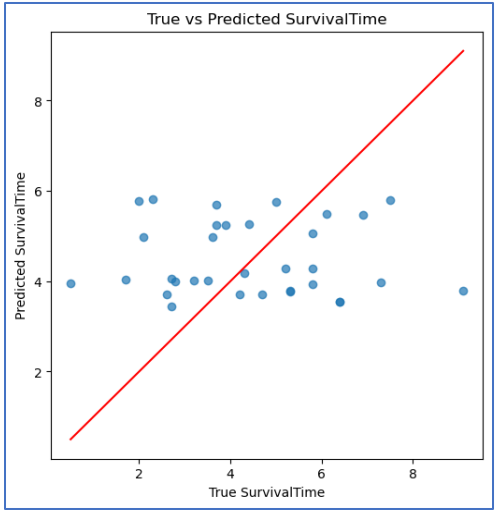
According to the metrics displayed in the previous slide, *XGBoost* was the most promising model.

However, it ended up underperforming in Kaggle, likely due to overfitting, even as we tried to tune some hyperparameters to prevent it.

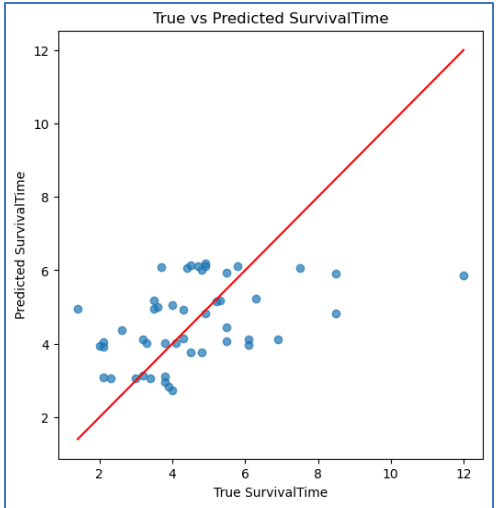
Our best submission, and hence with our best model, was with the *CatBoost* regressor, also with some of the parameters tuned, to ensure accuracy but also to prevent overfitting: max_depth = 1 and learning_rate = 0.01.

The y-y_hat plot on the lower right corner is correspondent to the *CatBoost* regressor's performance. In that plot, many of the predictions align much closer to the diagonal line, reflecting more accurate predictions of survival times.

This was to be expected already, as this model has displayed a significantly lower mean error and standard deviation than the baseline, as displayed in the table in the previous slide.



Baseline plot



CatBoost plot

Task [4.1]

Evaluation against the baseline (before Isomap):

As observed in the table and as expected, both the model for task 3.1 and for the current one significantly outperform the baseline in the most meaningful metrics, presenting a much lower mean error and standard deviation.

The current model is an improvement over the one of task 3.1, which suggests that including both labeled and unlabeled data when it comes to imputation does improve the model's robustness. However, this improvement, meaningful as it is, is also very small.

Alongside the application of the Isomap, we also tried performing PCA and even tuned its number of components to the optimal value. However, the results were only getting worse with PCA, so we dropped it.

Model	Max error	Min error	Mean error	Std Dev
Baseline	5.309497	0.122906	1.739313	2.086137
Task 3.1	6.050910	0.035233	1.296114	1.693560
Task 4.1	6.084725	0.039101	1.284201	1.685576

Task [4.2]

Evaluation against the baseline:

Looking at the metrics in the table below, once again the baseline model is significantly outclassed by all the other models.

We had locally determined in task 4.1, that for a linear regression, the best number of components for the Isomap is 4. And in task 4.2, we had determined that number of components that minimizes the cross-validation error for the CatBoost model is 3.

Though the model for task 4.2 performs decently compared to the baseline, the model for task 4.1 that was just a linear regression without applying the optimal Isomap, still outperforms it. By very little when it comes to mean error, but by quite the significant amount regarding standard deviation, indicating the current model may be less consistent. Maybe it has overfitted the training dataand can't generalize as well to the test set, where these metrics are generated from.

Along these lines, for task 4.1, the application of the Isomap has degraded the performance somewhat, most likely due to the loss of information during dimensionality reduction. It appears that the Isomap may not be properly capturing the patterns of the data.

Model	Max error	Min error	Mean error	Std Dev	Test Score	Kaggle
Baseline	5.309497	0.122906	1.739313	2.086137	4.35506	3.44036
Task 4.1 (w/o Isomap)	6.084725	0.039101	1.284201	1.685576	2.41897	-
Task 4.1 (w/ Isomap(4))	6.570791	0.018892	1.320009	1.794507	2.79078	3.24221
Task 4.2 (CatBoost w/ Isomap(3))	6.290252	0.032914	1.282456	1.778130	2.69446	3.07344

Task [4.2]

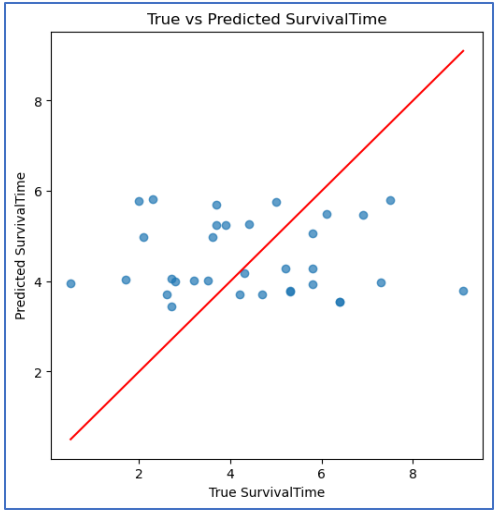
Evaluating the y-y hat plot against the baseline:

After imputing the data with both labeled and unlabeled information, and fitting a 3 component Isomap to it, the best model from task 3.2 was also the most performant in task 4.2, a CatBoost regressor with parameters: max_depth=1 and learning_rate=0.01.

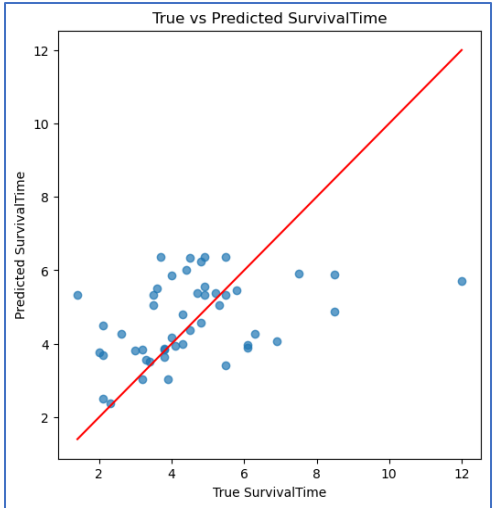
The plot for the CatBoost model displays a relatively tight clustering of predictions around the red line, especially for low to mid-range survival times. This is a clear sign of much greater accuracy than the baseline model, as there is a very reduced spread around the diagonal.

However, there are still some outliers, which are particularly more noticeable at higher survival times.

In conclusion, as was already expected from the table in the previous slide, the current model far outperforms the baseline in predictive power, especially when coupled with an Isomap.



Baseline plot



CatBoost plot

Task [5.1]

In our final model, we decided on experimenting with the Stacking Regressor (with 5-fold cross-validation), hoping to benefit from the diversity in results from some of the different regressors we had used during the assignment across the various tasks, which all have their own strengths. The whole point was to create a model much more capable of generalization than the ones that came before.

Up until this point, the most successful results we had gotten, and our best submission so far was the one of task 4.2, in which we had imputed the missing data with a *KNN* imputer of K=19 using both labeled and unlabeled data points, standardized it, and then used an Isomap to reduce the data to only the most meaningful dimensions.

This sort of pre-processing enhanced the model's ability to capture meaningful information and generalize to unseen data, and so we decided to bring the very same pre-processing steps to task 5, our ultimate model.

As for the base estimators used in the stacking regressor, we wanted them to capture multiple different aspects of the data, and as such, should present at least some diversity and complement each other.

Our choice of base regressors was: *Random Forest*, *CatBoost*, *XGBoost*, *KNN* and *Ridge*.

By including the most performant gradient boosting models, an ensemble model, a localized learner and a simpler model with regularization, our main stacking regressor is able to capture both linear and non-linear complex relationships as well as localized patterns while preventing overfitting.

As for our final estimator, we chose a simple *Ridge* regressor to linearly combine all the predictions of the base models and prevent overfitting.

Optimizing the parameters for the used regressors involved a lot of trial and error, but for the boosting and tree-based models we mostly kept the optimal parameters of task 3.2's models and just experimented with the value for *n_estimators*, aside from the *Random Forest* regressor in which we significantly decreased the *max_depth* attribute, which was what made the biggest difference in our final model's performance.

As for the K and α values for the *KNN* and *Ridge* regressor, respectively, we plotted how these values impact the model's performance, trying to reach an equilibrium between a low error and also preventing overfitting to the data.

While we use a *Ridge* regressor as both the meta-model and as part of the base models, in the final estimator the value for α is low, as it is only intended to combine the predictions of the base models without overly constraining their results. The α for the base model *Ridge* is much higher as its purpose there is to capture the simplest patterns in the data and avoid overfitting.

Task [5.1]

Evaluation against the baseline and task 4.2:

To no surprise at this point, the current model greatly outperforms the baseline model in mean error and standard deviation.

However, in terms of pure metrics, especially mean error and deviation, the model from task 4.2 seems to slightly surpass the current one, possibly indicating more consistency in predictions than our newest model;

And yet, we argue that the current model is the better one, as it not only significantly outperforms the one developed in task 4.2 on Kaggle but also shows a much smaller discrepancy between the local test score and the Kaggle score of only about 0.05, compared to the discrepancy of 0.39 points of task 4.2.

This leads us to believe that despite being marginally outclassed by 4.2 in terms of metrics, task 5.1's model can generalize better to unseen data and is less likely to overfit the local dataset, making it more reliable in general.

For this task, we will be omitting the y-y_hat plot as the analysis would essentially consist in parroting what was already stated in previous tasks, since the local test score (from which the plot is created) for the current model is relatively similar to that of other tasks, 4.2 being one of them. This to say that the plot analysis in task 4.2 is just as valid for this current task and the reader may simply refer to it. It's hard to notice any significant changes in the plot.

Model	Max error	Min error	Mean error	Std Dev	Test score	Kaggle
Baseline	5.309497	0.122906	1.739313	2.086137	4.35506	3.44036
Task 4.2	6.290252	0.032914	1.282456	1.778130	2.69446	3.07344
Task 5.1	6.084725	0.054314	1.325969	1.826762	2.82443	2.87235

What went wrong

Having implemented more than half of the assignment, we had to change the random state in the train-test split, as the one we had, displayed huge discrepancies between the local test score and the Kaggle score, indicating that the split had to be refactored. Due to this, we had to re-analyze everything that had been done up to that point, since now quite a few scores would be different and so would be the optimal parameters.

We followed the tutorial for using the Accelerated Failure Time CatBoost regressor in task 3.2, but the results were quite unsatisfactory and couldn’t even be used.

The daily submission limit in Kaggle was a hindrance, as we had to wait for several hours until we could properly progress with the assignment.

Unnotified changes to the assignment description in the earlier tasks made us have to go back to them after already having drawn our conclusions and moved on. These changes forced us to re-analyze the results and led to different optimal models and parameters, affecting the later tasks that were dependent on conclusions drawn from the earlier ones.

What went great

Through plenty of trial and error, as well as some sleepless nights, we have acquired further practical competence on what it takes to train a machine learning model and on how to evaluate it.

We also felt somewhat more comfortable doing so in this second assignment, due to previous “experience” gained in the first.

We had the liberty to experiment with several different regressors and methods, of varying complexities, and it was interesting to observe how these affected performance.

Our code is competently implemented and organized, with comprehensive sections and subsections for each individual task .

Finally, we're pleased with the results of our final model, as it displayed a very noticeable evolution in performance over the models developed in the compulsory tasks.